# CPU (Serial/OpenMP) vs GPU (CUDA) Sorting: Design and Performance Report

March 28, 2026

## Authors and Contributions


| Name | B.N | Contributions |
|---|---|---|
| Mohamed Toukhy Mohamed | 28 | OpenMP merge sort; benchmark runner; plotting pipeline; infrastructure core. |
| Ali Hossam Ali | 20 | CUDA bitonic sort; plotting pipeline; infrastructure core. |
| Omar Ahmed Abdelaziz | 22 | Serial merge sort baseline; report; project structure; infrastructure core. |


## Abstract


The report present a performance and engineering comparisons of three sorting approach implemented within one benchmarked application:

(1) serial CPU baseline using a recursive merge sort

(2) multi-core CPU parallel merge sort using OpenMP task parallelism with a tunable cutoff for task granularity

(3) CUDA GPU bitonic sorting network with a safe power-of-two padding for arbitrary input length.

The goals are to make a scaling with input size, quantify multi-thread CPU speedup, evaluate GPU tuning sensitivity (block size) and interpret the role of overheads such as synchronization and host–device data movement. Results are based on the measured dataset in `results/results.csv` and the plots generated by `plot.py` at `results/`.


## 1. Introduction

Sorting is a foundational primitive in systems and performance critical pipelines it appears in database query execution, data analytics, scientific computing and general-purpose applications. Because it is sensitive to compute and memory behavior and sorting provide a useful case study for understanding parallel performance on modern heterogeneous hardware.

This project is designed to make CPU vs GPU tradeoffs measurable and explainable. It integrate multiple implementations behind one experimental driver, ensure correctness checking on every run, logs results in a consistent schema and generate standardized plots that highlight scaling, speedup, and tuning effects.

## 2. Objectives and Deliverables

1. Provide different implementations spanning serial CPU, parallel CPU and GPU execution model.
2. Measure performance under controlled variability using repeat trials and multiple input distribution.
3. Separate tuning knobs relevant to each model (thread count and task cutoff in CPU block size on GPU).
4. Produce publication-style figure summarizing scaling and tuning behavior.

Deliverables:

- A unified benchmarking application (`bin/sort_app`) that generate input runs each implementation and verifies correctness and records averaged timings.
- A results dataset (`results/results.csv`) with consistent column for analysis.
- A figure set (`results/plot_a.png`, `results/plot_b.png`, `results/plot_c.png`) summarizing the most important comparisons.

## 3. Platform Summary

The recorded platform information (see `results/system_info.txt`) indicates:

- **CPU:** AMD Ryzen 7 5800H (16 logical CPUs)
- **GPU:** NVIDIA GeForce RTX 3050 Laptop GPU (4 GB)
- **Toolchain:** GCC 13.3.0 and CUDA 12.0

## 4. Experimental Methodology

### 4.1 Input generation and distributions

The benchmark generate integer arrays using a seeded pseudo random generator and support four distributions:

- **Uniform:** values drawn uniformly from the integer range $[0,10{,}000{,}000]$.
- **Gaussian:** values drawn from a normal distribution with $\mu = 5{,}000{,}000$ and $\sigma = 1{,}000{,}000$ (then cast to integers).
- **Nearly sorted:** uniform array is first sorted ascending, then perturbed by ~1% random swaps.
- **Reversed:** uniform array sorted descending.

This distributions are important because real workloads are rarely uniformly random; many datasets are partially ordered or structured.

### 4.2 Repeats, averaging, and correctness

Each configuration is executed multiple times (five repeats in the recorded dataset). A new input is generated per repeat by incrementing the base seed.

Correctness is enforced on every repeat by sorting a reference copy with `std::sort` and performing an element-wise comparison against the candidate result. This catches silent logic errors that can occur in parallel implementations.

### 4.3 What the reported time represents

The dataset's primary metric is **end-to-end average runtime in milliseconds**, recorded as `AvgTime(ms)`.

- For CPU implementations (serial and OpenMP), runtime is measured with a monotonic wall-clock timer.
- For the GPU implementation, `AvgTime(ms)` is also end-to-end wall-clock time (including padding, allocation, transfers, kernel launches and synchronization). The CUDA implementation additionally measures H2D/compute/D2H using CUDA events and prints a breakdown for interpretation, but the plots use end-to-end times for fair comparison.

## 5. Implementation Overview

### 5.1 Serial CPU baseline (recursive merge sort)

The serial baseline use a recursive merge sort with a temporary buffer. This provides a deterministic CPU reference.

### 5.2 OpenMP task-parallel merge sort

The OpenMP implementation follows a standard divide-and-conquer merge sort:

- Split the array into two halves.
- Sort each half in parallel as OpenMP tasks.
- Wait for both tasks.
- Merge the results using a temporary buffer.

A key tuning parameter is a **cutoff threshold** controlling task granularity. When a subproblem size falls below the cutoff, the implementation falls back to a serial sort of that segment to avoid excessive task overhead.

### 5.3 CUDA bitonic sorting network

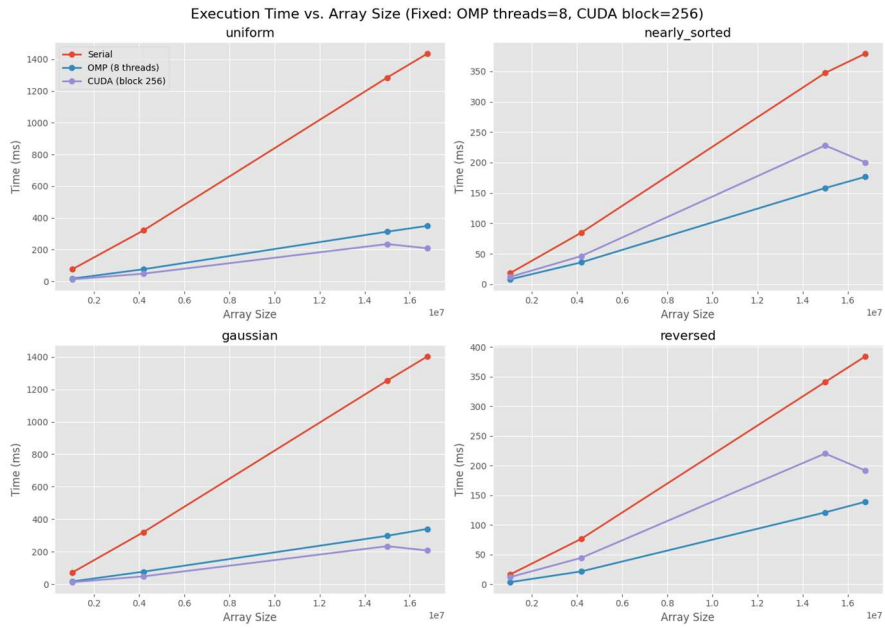
The GPU implementation uses a bitonic sorting network. Because bitonic networks naturally target power-of-two length the code pads non-power-of-two arrays up to the next power of two using a sentinel value (`INT_MAX`). For ascending sort, sentinel values migrate to the end. the result is then truncated back to the original length.

The implementation synchronizes after each stage launch, which improves determinism and makes stage boundaries explicit  but can increase overhead relative to more asynchronous GPU pipelines.

# 6. Results

This section discusses the three figures generated from the measured dataset.

## 6.1 Figure A: Execution time vs input size (all distributions)



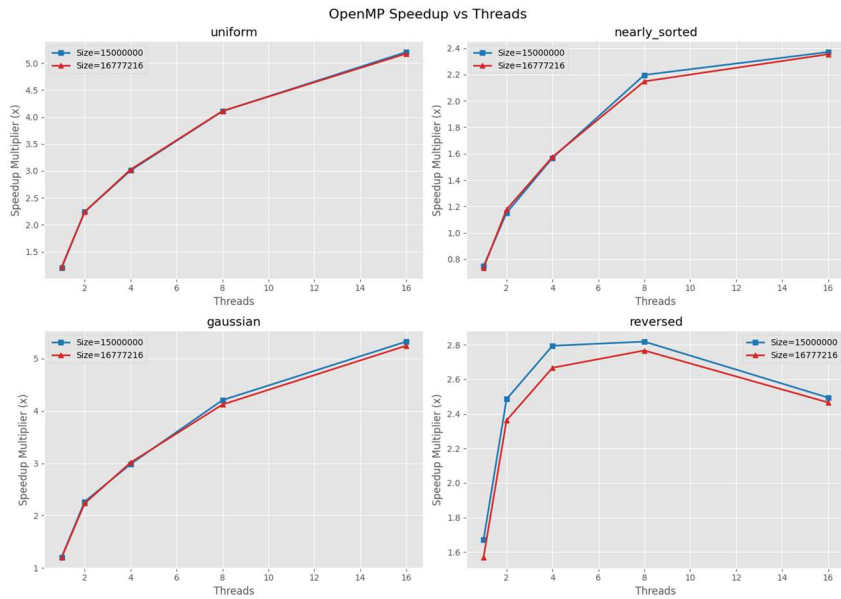

Each subplot compares **Serial**, **OpenMP (fixed at 8 threads)**, and **CUDA (fixed at block size 256)** timing for one distribution.

Headline values for the **uniform** distribution (end-to-end `AvgTime(ms)`) include:

- At **16,777,216 elements**, Serial averages **1435.82 ms**, OpenMP (8 threads) averages **349.38 ms** and CUDA (block size 256) averages **207.93 ms**.
- At **4,194,304 elements**, OpenMP (8 threads) averages **75.79 ms**, while CUDA (block size 256) averages **48.13 ms**.

These values show that both CPU and GPU parallelism are beneficial at large sizes on this platform.

## 6.2 Figure B: OpenMP speedup vs thread count



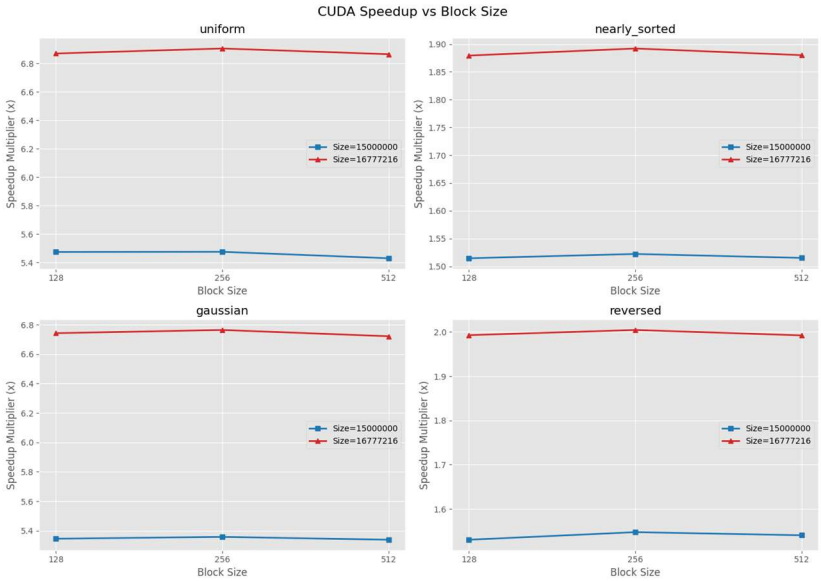

Each subplot plots speedup versus the serial baseline for the **two largest input sizes** in the dataset (so both a power-of-two size and a non-power-of-two size are visible).

Using the recorded **uniform** timings for **16,777,216 elements**, the speedups vs the serial baseline are:

| Threads | OpenMP time (ms) | Speedup vs serial |
|---|---|---|
| 1 | 1178.41 | 1.22× |
| 2 | 640.80 | 2.24× |
| 4 | 474.64 | 3.03× |
| 8 | 349.38 | 4.11× |
| 16 | 277.47 | 5.17× |

This shows strong scaling from 1 → 8 thread with diminishing returns up to 16 threads (typical of merge- and memory-bandwidth-heavy workloads).

## 6.3 Figure C: CUDA speedup vs block size




Each subplot plots CUDA speedup versus the serial baseline across block sizes (128, 256, 512), for the **two largest input sizes**.

For **16,777,216 elements** under uniform input CUDA achieves approximately:

- **6.87×** speedup at block size 128 (208.99 ms)
- **6.90×** speedup at block size 256 (207.93 ms)
- **6.86×** speedup at block size 512 (209.14 ms)

In this dataset and implementation, block size has only a modest effect at the largest sizes (the end-to-end runtime includes transfers and per-stage synchronization).

# 7. Discussion

## 7.1 Scaling and where parallelism helps

End-to-end results show that parallelism becomes increasingly beneficial as problem size grows. At the largest size tested (16,777,216 elements), OpenMP (16 threads) reaches about a **5.17×** speedup over the serial baseline (uniform), while CUDA reaches about a **6.90×** speedup at block size 256.

## 7.2 Sensitivity to input distribution

CPU baselines can benefit significantly from structured inputs (nearly sorted or reversed), while the GPU sorting network's runtime remains comparatively stable across distributions. This aligns with the algorithms' behavior: merge-based CPU work can become cheaper when partial order reduces comparisons, while a sorting network executes a fixed compare–swap pattern.

### 7.3 Tuning parameters and overheads

Two primary tuning axes emerge:

- **CPU tuning:** thread count and cutoff threshold (task granularity). Too-small cutoffs can create excessive task overhead; too-large cutoffs can reduce parallelism.
- **GPU tuning:** block size (and implicitly the synchronization/launch structure). For large sizes in this dataset, block size changes performance modestly, suggesting end-to-end runtime is also influenced by overheads outside pure compute.

## 8. Limitations and Assumptions

- Bitonic sorting networks are not asymptotically optimal compared to typical $O(n\log n)$ comparison sorts; performance relies on parallel throughput.
- Per-stage synchronization in the GPU path improves determinism and reduces throughput.
- Non-power-of-two inputs are padded to the next power of two, so GPU work can exceed the nominal input size.

## 9. Conclusion

This project provides a coherent comparison of sorting across three execution models: serial CPU merge sort, OpenMP task-parallel merge sort, and CUDA bitonic sorting. The measured results show that OpenMP delivers substantial speedup when multi-threading is applied to sufficiently large sizes, and that the CUDA implementation achieves the best end-to-end time at the largest tested size on this platform. The included figures make the central tradeoffs visible: CPU scaling depends on task overhead and memory behavior, while GPU performance depends on launch structure, synchronization strategy, and host–device overheads.